Multi-Level NAND and NOR Circuits

CENG 2112.01 Lab for Digital Circuits

Submitted by

David Dulaney

2344357

Computer Science

University of Houston-Clear Lake

Houston, Texas 77058

Apr 7,2026

Contents

Abstract

A NAND gate is a device that requires two inputs based off two options which are 0 and 1, then it will produce one output which is inverted. If the inputs of this gate are the same (ex. Both inputs are 1), then the output will be the same as the inputs inverted. Also, if one of the inputs are 0, rather it producing a 0, it will invert and produce a 1. A NOR gate is a device that requires two inputs based off two options similar to the NOR gate and follows the same rules as far as the output produced will be inverted based on the two inputs, but they are opposite in the sense that if one of the inputs is a 1, then it will produce the inversion which is a 0.
The important finding in this experiment is you can replace inverters in between NAND or NOR gates with a NAND or NOR gate depending on the circuit you are creating.

Introduction

Similar to De Morgan's Theorem, NAND and NOR gates work off inverting an expression multiple times to conclude to the same outcome of the original expression but with a two level circuit to make sure every input is delayed with the same amount of time. This is important because if you have a circuit that is late to getting an input because of a gate before it waiting on inputs. Then the whole circuit's output is delayed.

Requirements

The requirement for this lab is to simulate a multi-level NAND and NOR circuit taken from a circuit mixed with AND and OR gates. Then we used a logic analyzer within Multisim to check the waveforms of different outputs (1 and 0. We also coded a schematic in Vivado using VHDL as well as to have a testbench of the waveforms to match the results from Multisim

Prelab

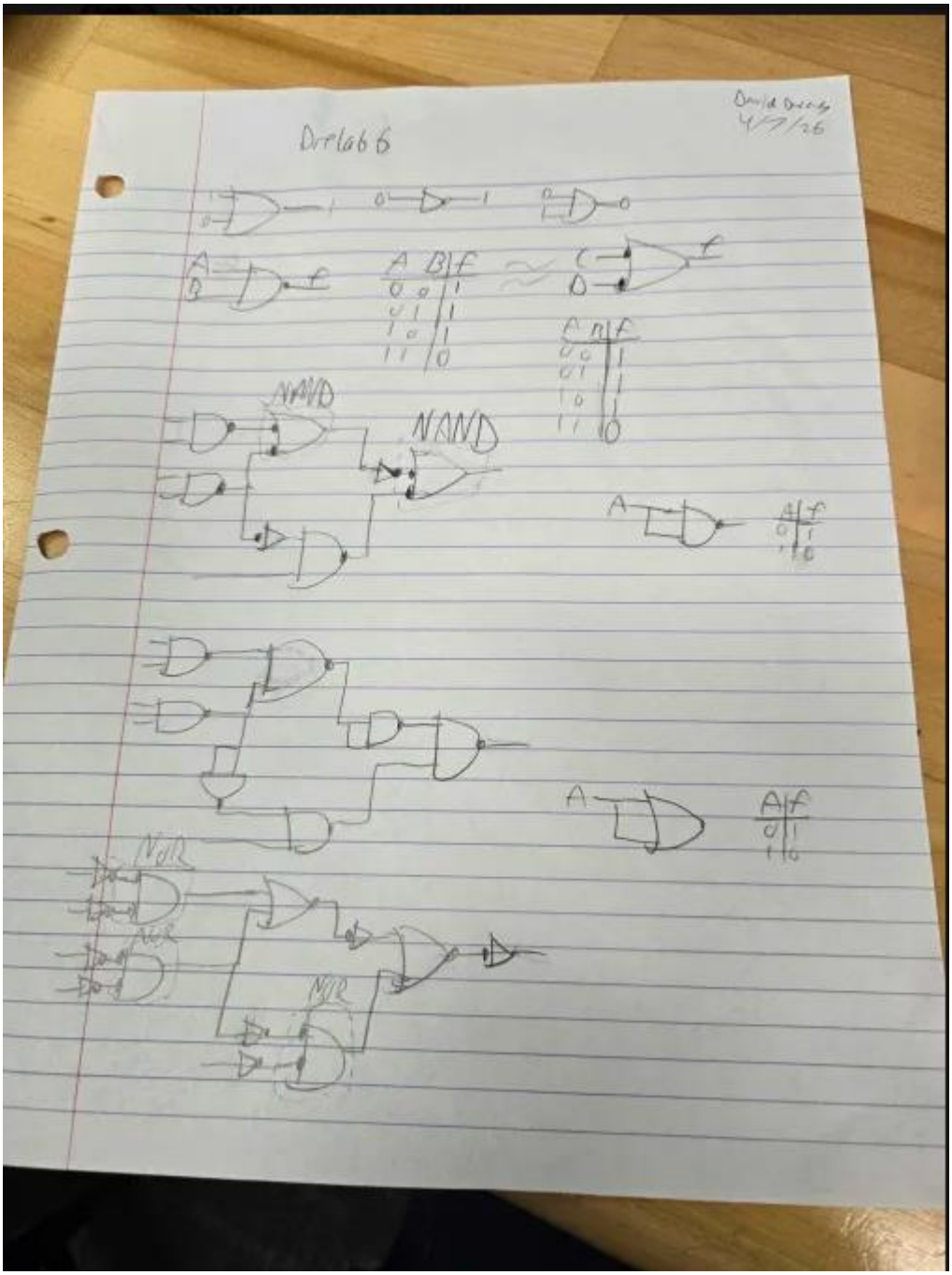

figure 1: prelab for multi-level NAND and NOR gate circuit

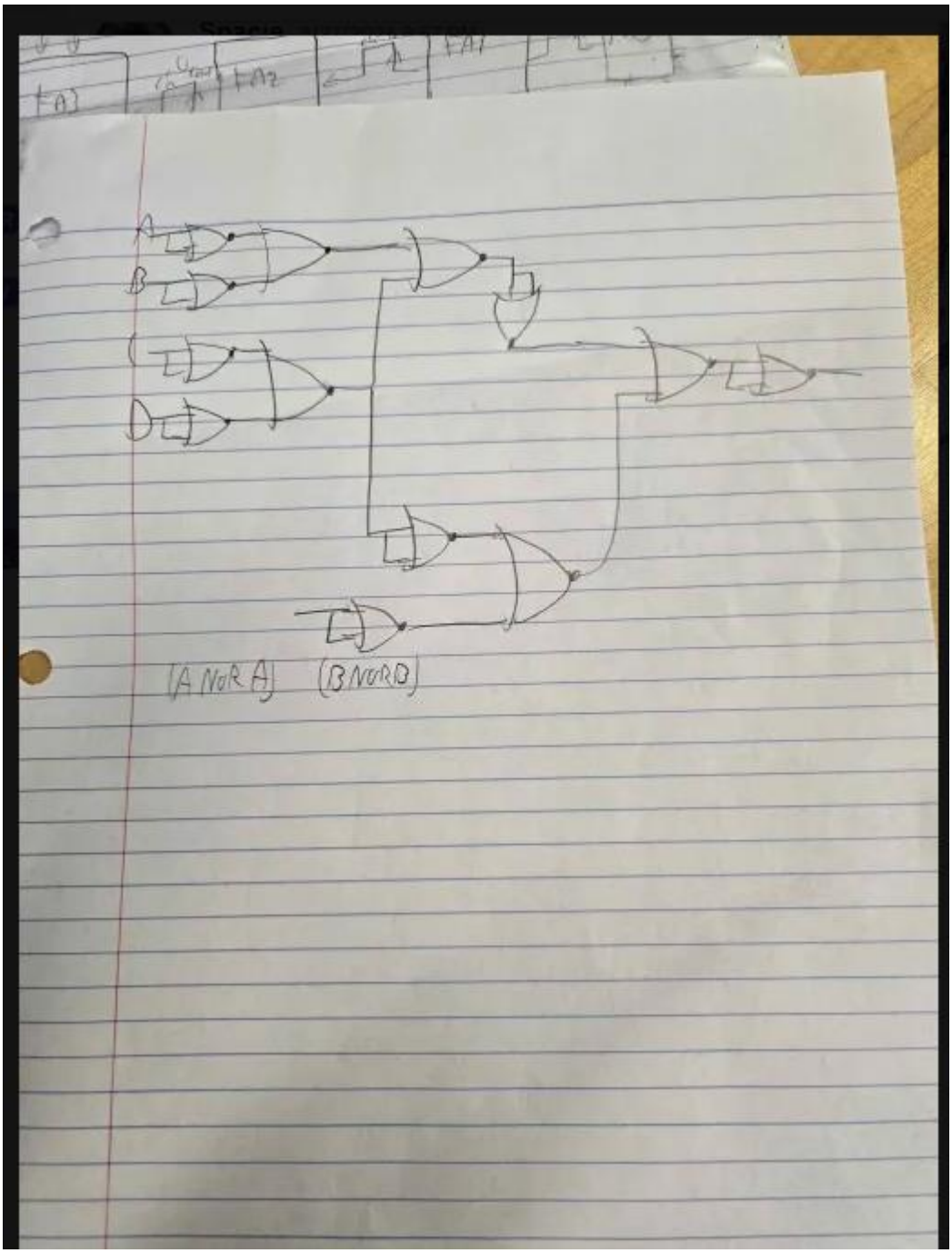

figure 1.1: prelab for multi-level NAND and NOR gate circuit part 2

Implementation

The first step in implementation was to build a schematic with a word generator and a logic analyzer with a NAND and NOR circuit based on five combinations of 0s and 1s to produce the same outputs. Then we simulated these combinations in VHDL to see if we get the same outcome as our schematics.

Design Code/Design Diagrams

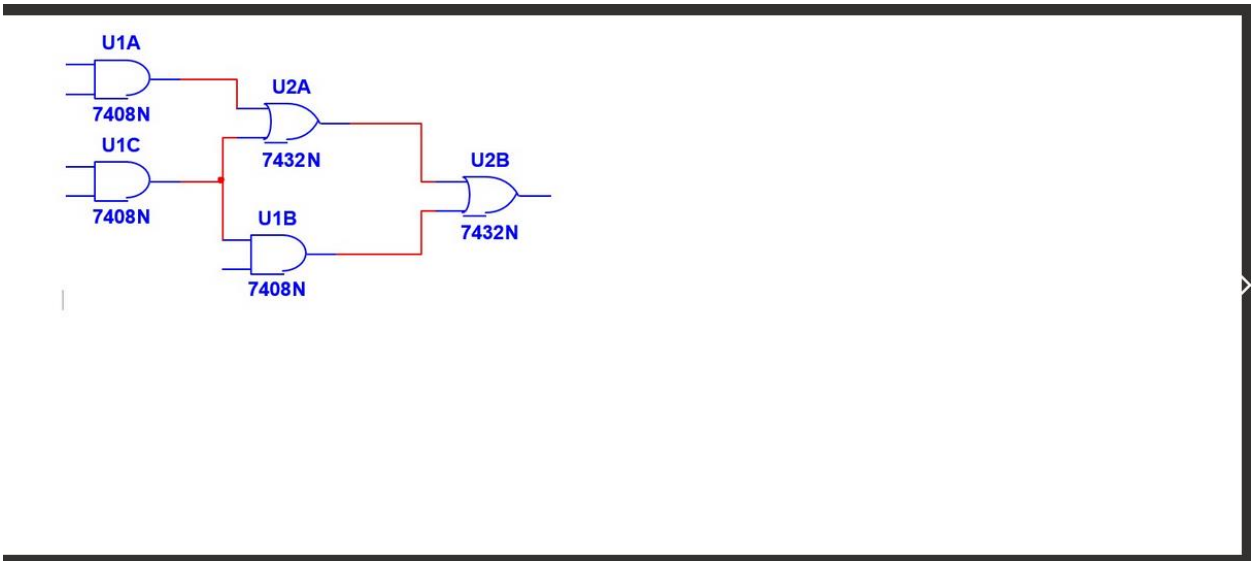

Figure 2: Original circuit with a mix of AND and OR gates

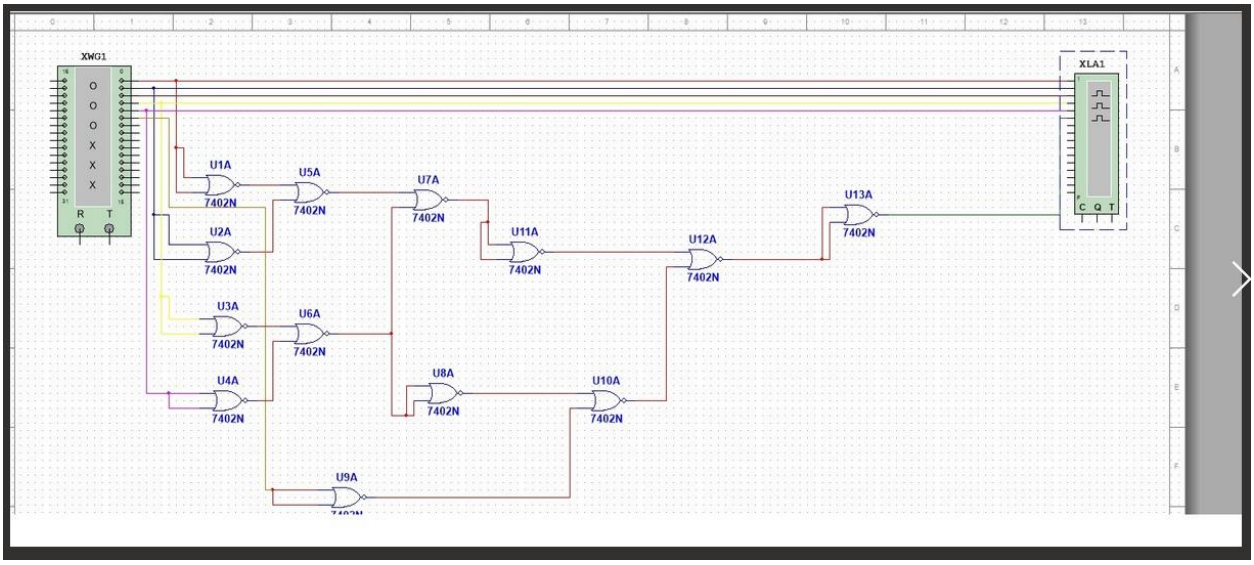

Figure 2.1: The original circuit expressed with NOR gates

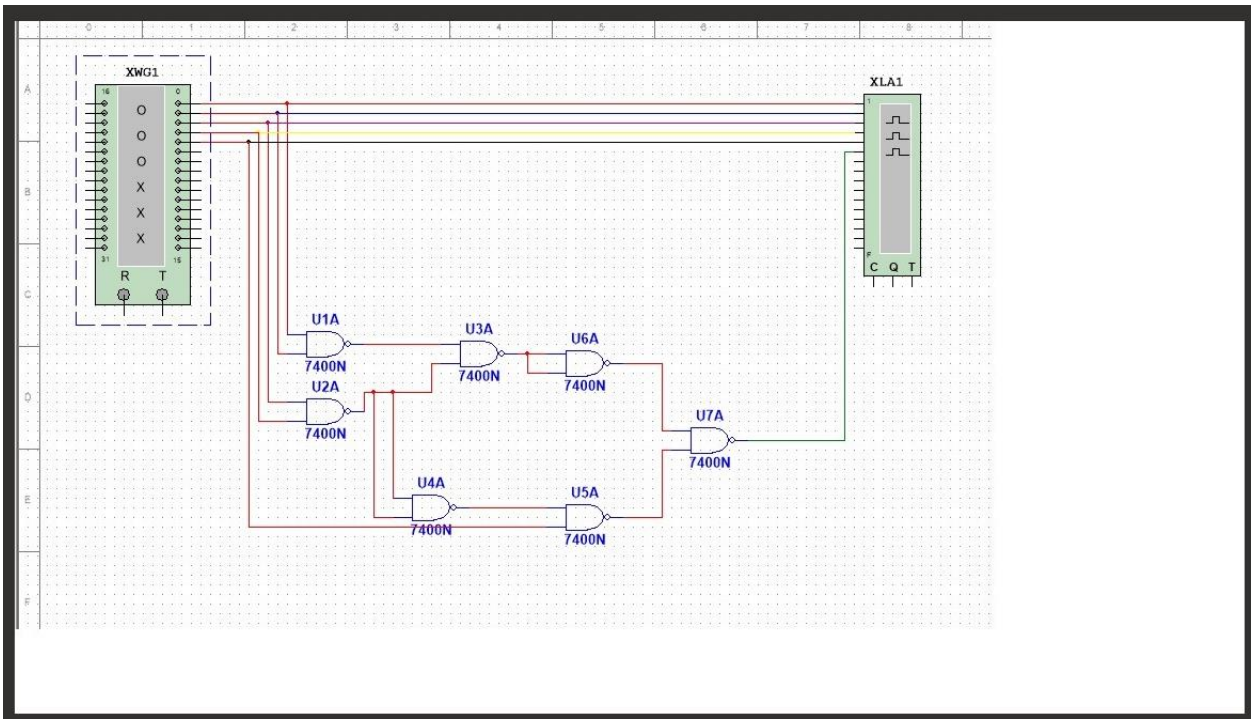

figure 2.3: The original circuit expressed with NAND gates

We concluded with these designs because they are the NAND and NOR versions of the original circuit we were given. The first schematic is the original circuit for, the second schematic is the NOR version of that circuit along with the third schematic being the NAND version of that circuit. Similar to De morgan's theorem on an expression, the NAND and NOR circuits will produce the same output as the original citcuit. This is shown in figures 6.1-6.10.

-- Company:

-- Engineer:

--

-- Create Date: 03/31/2026 03:44:33 PM

-- Design Name:

-- Module Name: project_9_nor - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:

```
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity project_9_nor is
  Port ( a : in STD_LOGIC;
         b : in STD_LOGIC;
         c : in STD_LOGIC;
```

```
        d : in STD_LOGIC;
        e : in STD_LOGIC;
        f : out STD_LOGIC;
        g : out STD_LOGIC;
        h : inout STD_LOGIC);
end project_9_nor;

architecture Behavioral of project_9_nor is

begin

--inserted logic box for NAND circuit
f<=(((a nand b) nand (c nand d)) nand ((a nand b) nand (c nand d))) nand (((c nand d) nand (c nand d)) nand e);
g<= ((((((A nor A) nor (B nor B)) nor ((C nor C) nor (D nor D))) nor (((A nor A) nor (B nor B)) nor ((C nor C) nor (D nor D)))) nor ((((C nor C) nor (D nor D)) nor ((C nor C) nor (D nor D))) nor (E nor E)))) nor ((((((A nor A) nor (B nor B)) nor ((C nor C) nor (D nor D))) nor (((A nor A) nor (B nor B)) nor ((C nor C) nor (D nor D)))) nor ((((C nor C) nor (D nor D)) nor ((C nor C) nor (D nor D))) nor (E nor E))));
h<=(((a and b) or (c and d)) or ((a and b) and e));

end Behavioral;
```

Schematics

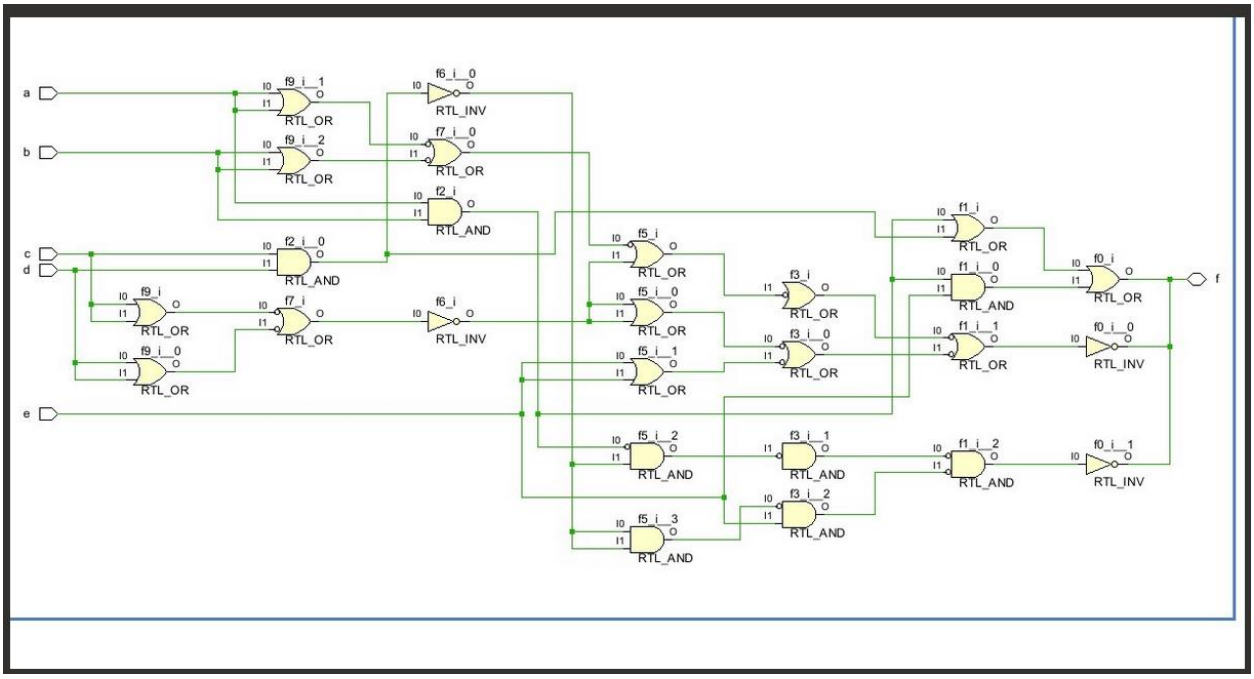

Figure 3: Schematic of all three circuits all having matching outputs

Testbench

```
----------------------------------------------------------------------------------

-- Company:

-- Engineer:

--

-- Create Date: 04/01/2026 11:41:43 AM

-- Design Name:

-- Module Name: project_9_TB - Behavioral

-- Project Name:

-- Target Devices:

-- Tool Versions:

-- Description:
```

```
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity project_9_TB is
  --  Port ();
end project_9_TB;
```

```
architecture Behavioral of project_9_TB is
--place chip on board
component project_9_nor is
  Port ( a : in STD_LOGIC;
         b : in STD_LOGIC;
         c : in STD_LOGIC;
         d : in STD_LOGIC;
         e : in STD_LOGIC;
         f : out STD_LOGIC;
         g : out STD_LOGIC;
         h : inout STD_LOGIC);
end component;
signal a, b, c, d, e: STD_LOGIC;
signal f,g,h:std_logic; --instantiating wires for each port
begin
UUT: Project_9_nor PORT MAP(a,b,c,e,f,g,h);

process
begin
--giving 33 combinations of inputs for each port
a<= '0'; b<= '0'; c<= '0'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '0'; e<= '1'; wait for 10ns;
```

```
a<= '0'; b<= '0'; c<= '1'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '0'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '0'; c<= '1'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '0'; d<= '1'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '0'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '0'; e<= '1'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '1'; e<= '0'; wait for 10ns;
a<= '0'; b<= '1'; c<= '1'; d<= '1'; e<= '1'; wait for 10ns;
```

```
end process;

end Behavioral;
```

FPGA board code for NAND and NOR circuits as well as the original circuit:

```
#assigning 5 ports as inputs

set_property PACKAGE_PIN V17 [get_ports a]

set_property IOSTANDARD LVCMOS33 [get_ports a]

set_property PACKAGE_PIN V16 [get_ports b]

set_property IOSTANDARD LVCMOS33 [get_ports b]

set_property PACKAGE_PIN W16 [get_ports c]

set_property IOSTANDARD LVCMOS33 [get_ports c]

set_property PACKAGE_PIN W17 [get_ports d]

set_property IOSTANDARD LVCMOS33 [get_ports d]

set_property PACKAGE_PIN W15 [get_ports e]

set_property IOSTANDARD LVCMOS33 [get_ports e]


#assigning 1 port as output

set_property PACKAGE_PIN U16 [get_ports f]

set_property IOSTANDARD LVCMOS33 [get_ports f]

set_property PACKAGE_PIN E19 [get_ports g]

set_property IOSTANDARD LVCMOS33 [get_ports g]

set_property PACKAGE_PIN U19 [get_ports h]

set_property IOSTANDARD LVCMOS33 [get_ports h]
```

Implemented design for NAND and NOR circuits as well as the original circuit:

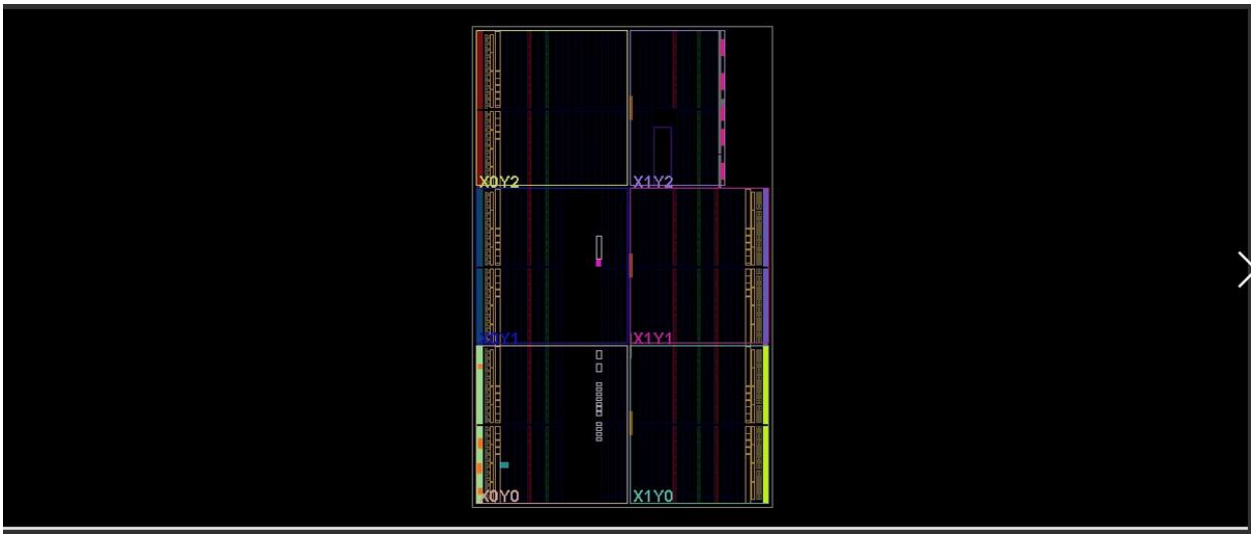

Figure 4: implemented design of the original circuit along with the NAND and NOR versions of it.

Waveform/Results

The waveforms in figures 5.1 and 5.2 represent The NAND and NOR circuits derived from the original circuit with the same outputs given the same inputs. In figure 5.1, the red line shows the voltage when both inputs are fluctuating between four combinations of 0s and 1s. Where each input is being sent through a NAND gate at different levels in the circuit, this is also shown in figure 5.2 with NOR gates The output is based on the green line, when inputted a combination of 1s and 0s the output will be 1 when the red and blue line are 1, otherwise it will output 0.

Figure 5.1: Waveform of the NAND circuit of the original circuit with five inputs

Figure 5.2: Waveform of the NOR circuit of the original circuit with five inputs

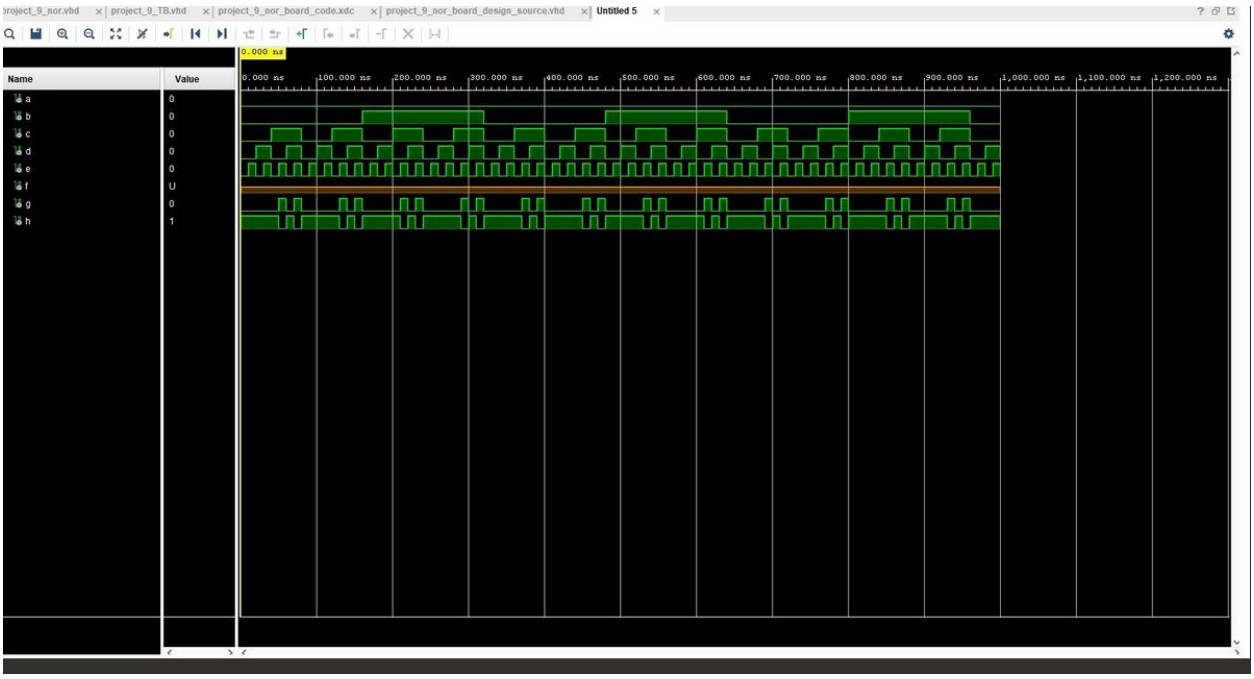

Figure 5: Behavioral simulation of the original circuit along with the NAND and NOR versions of the circuit.

The result in figure 6.1 shows the Arduino board for the NAND, NOR, and original circuit where 00001 is inputted, the output is 0 for all circuits.
The result in figure 6.2 shows the Arduino board for the NAND, NOR, and original circuit where 10001 is inputted, the output is 0 for all circuits.

The result in figure 6.3 shows the Arduino board for the NAND, NOR, and original circuit where 00011 is inputted, the output is 1 for all circuits.
The result in figure 6.4 shows the Arduino board for the NAND, NOR, and original circuit where 01111 is inputted, the output is 1 for all circuits.
The result in figure 6.4 shows the Arduino board for the NAND, NOR, and original circuit where 10111 is inputted, the output is 1 for all circuits.

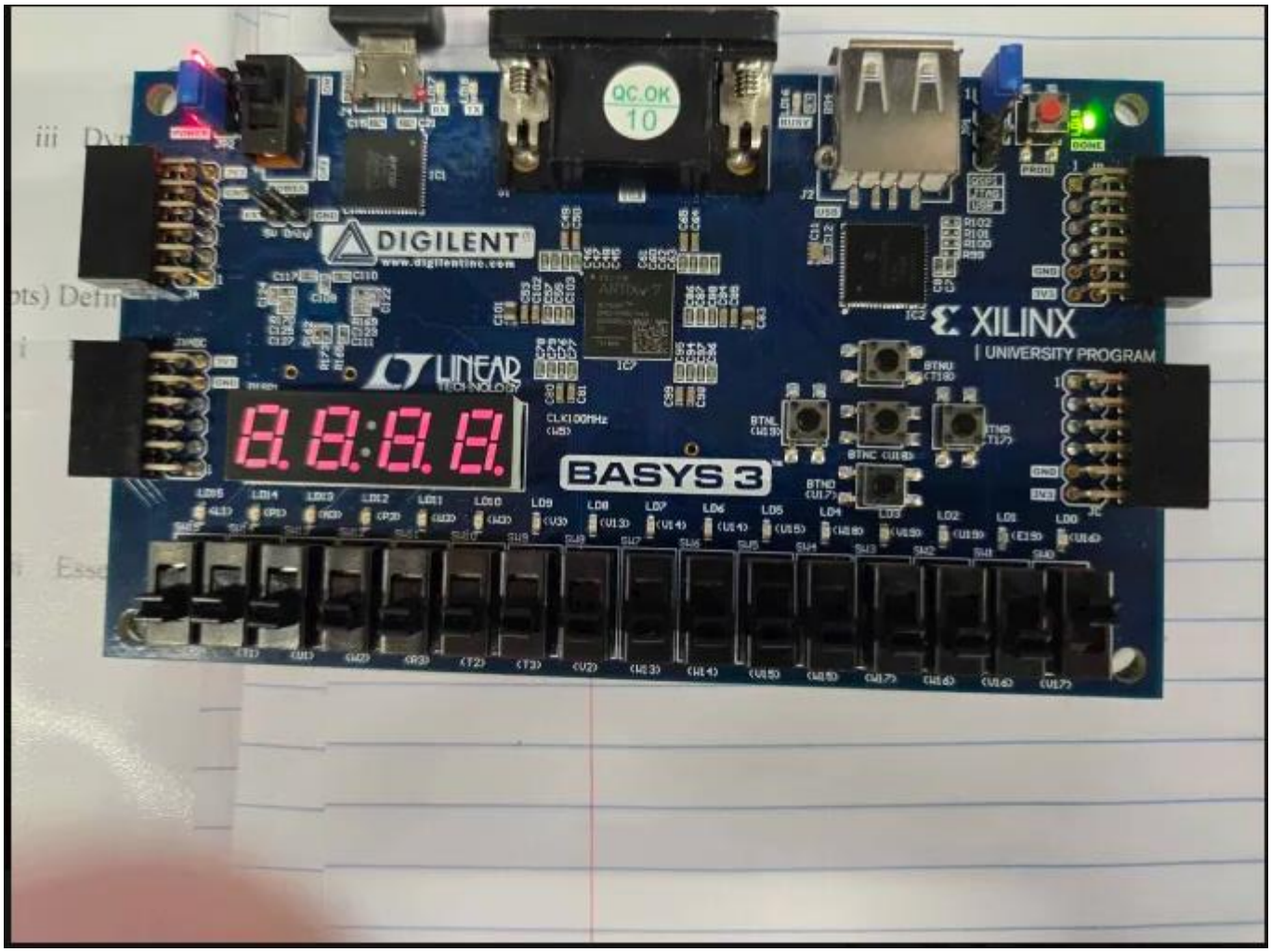

Figure 6.1: physical board showing the NAND, NOR, and original circuit with 00001 is the input and has an output of 0 for all circuits.

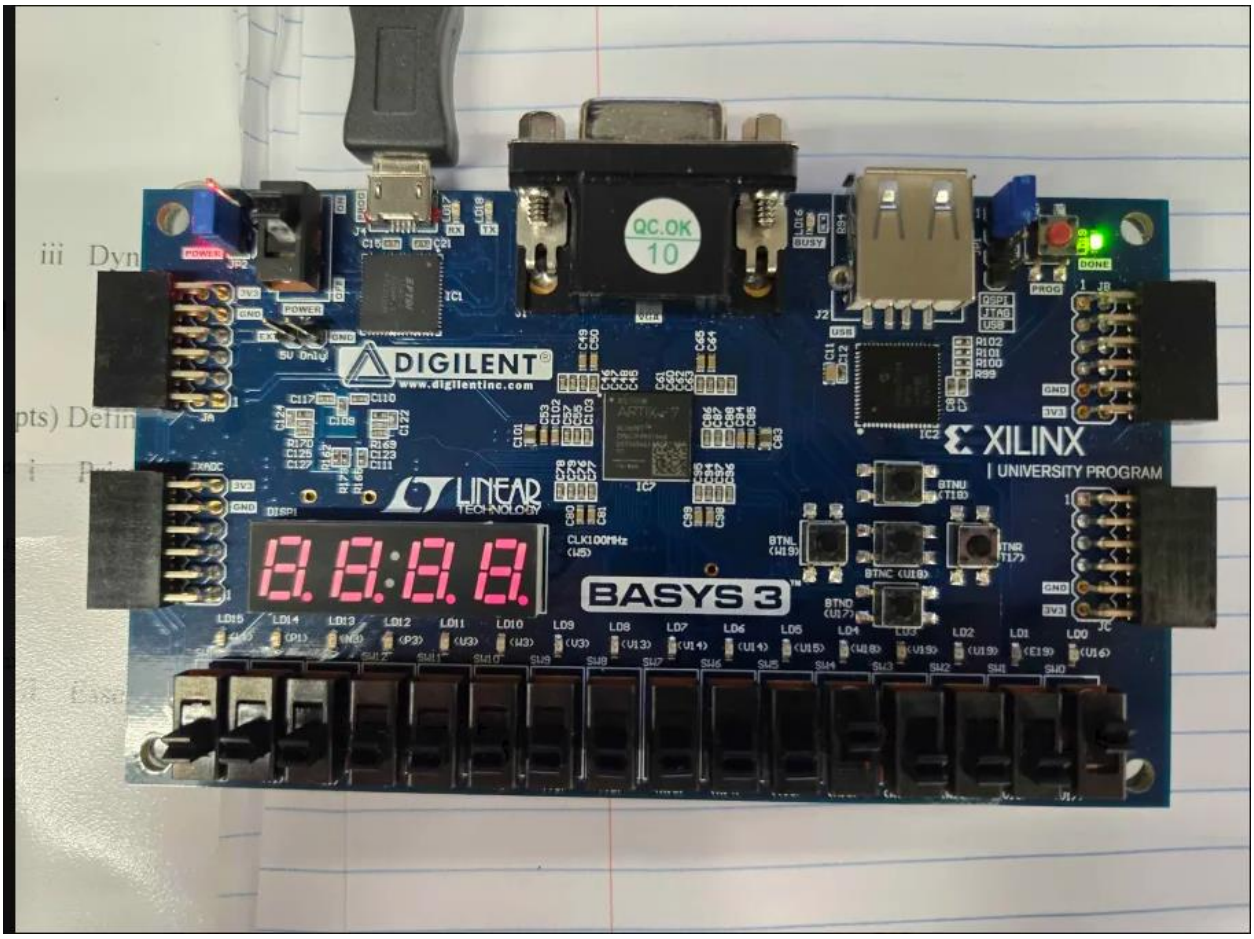

Figure 6.2: physical board showing the NAND, NOR, and original circuit with 10001 is the input and has an output of 0 for all circuits.

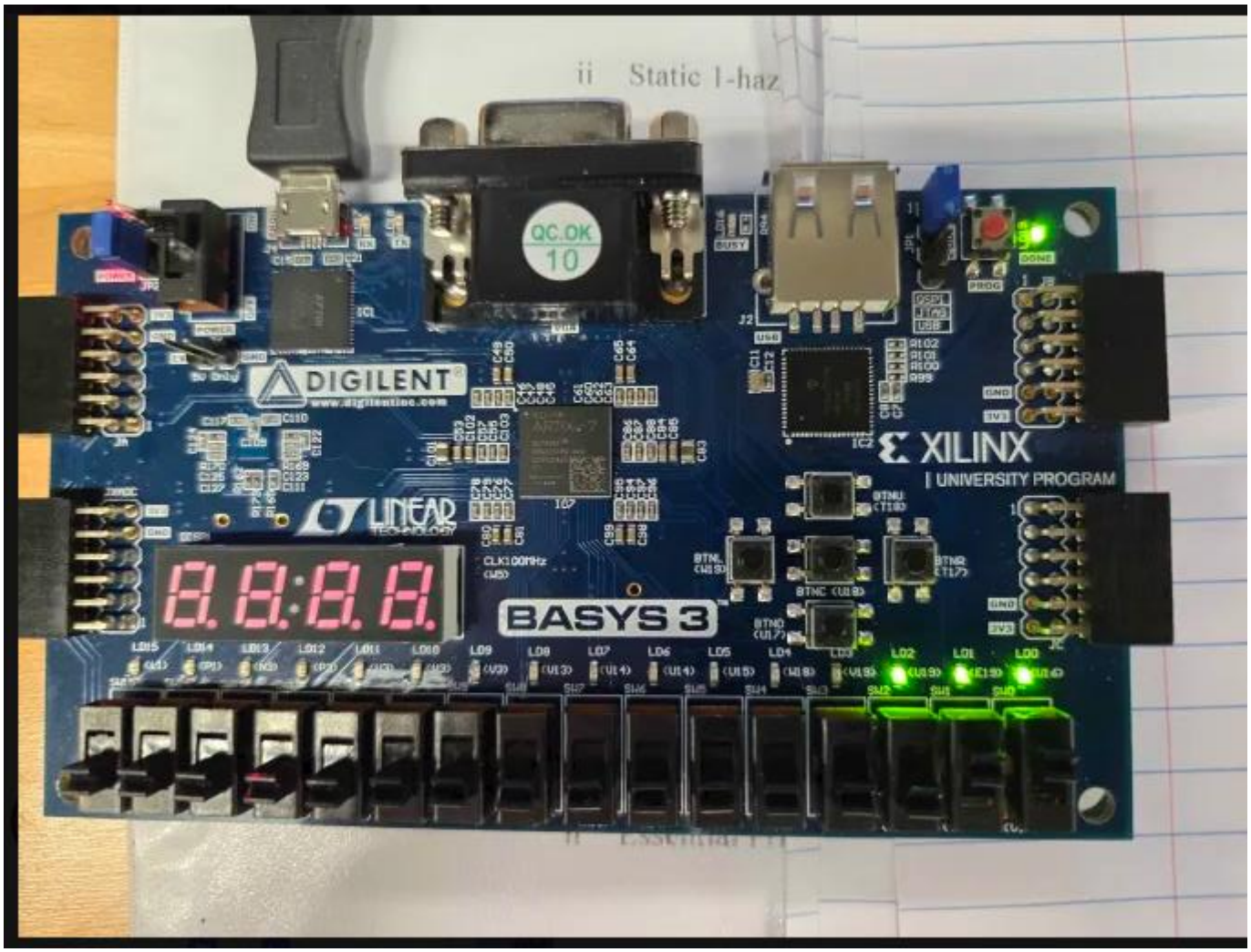

Figure 6.3: physical board showing the NAND, NOR, and original circuit with 00011 is the input and has an output of 1 for all circuits.

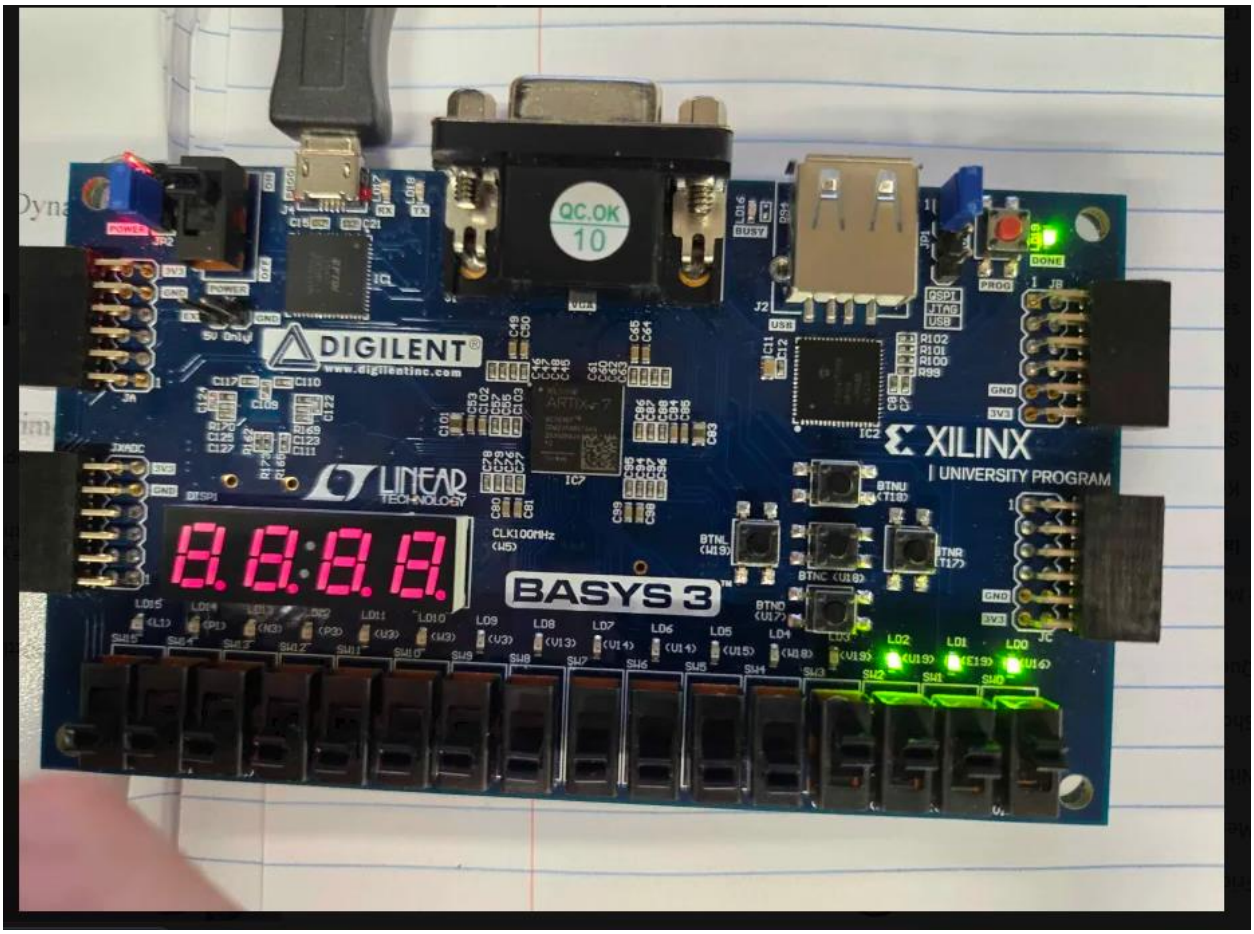

Figure 6.4: physical board showing the NAND, NOR, and original circuit with 01111 is the input

and has an output of 1 for all circuits.

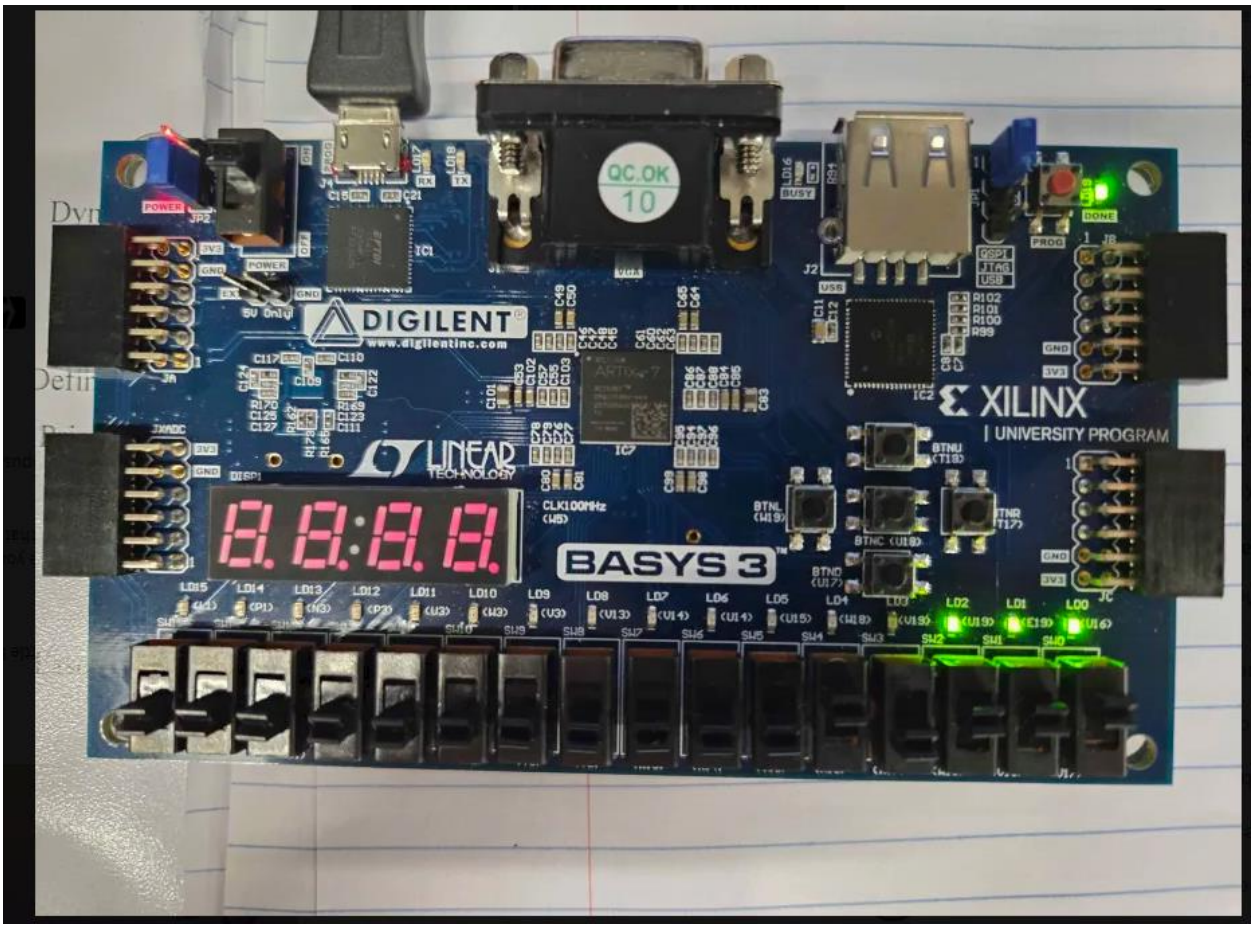

Figure 6.5: physical board showing the NAND, NOR, and original circuit with 10111 is the input and has an output of 1 for all circuits.

Conclusion

This project was designed to understand that a NAND and NOR circuit can be created from a circuit with an AND-OR circuit. The main takeaway is that using de morgan's law can manipulate an AND-OR circuit to a NAND or NOR circuit which can help with delay timing throughout the circuit, the only disadvantage to this is the cost of the wires and gates needed to complete these circuits.